

Forecast-Driven Capacity Reservation and Right-Sizing for Cloud Cost Optimization under Intermittent Random Demand Surges

First Author, Second Author, Third Author, Fourth Author

Faculty of Computer and Information Sciences

Ain Shams University

Cairo, Egypt

{author1, author2}@cis.asu.edu.eg

Supervisor Name

Faculty of Computer and Information Sciences

Ain Shams University

Cairo, Egypt

supervisor@cis.asu.edu.eg

Abstract—Enterprise cloud spending continues to grow, yet a large fraction is wasted on over-provisioned and mispriced resources. The difficulty is sharpest when a steady base demand is punctuated by *intermittent random demand surges*, which force operators either to over-reserve capacity year-round or to absorb spikes at expensive on-demand rates. We present Smart Cloud Optimizer, an end-to-end system that couples time-series demand forecasting with optimization-based capacity reservation to minimize cloud cost while preserving headroom for surges. The system (i) collects cost and utilization telemetry from Amazon Web Services, (ii) forecasts daily demand using an ensemble of five models with anomaly-aware preprocessing, (iii) separates demand into a *base* component, served by committed reserved instances, and a *surge* component, served by elastic on-demand headroom sized at the 95th percentile of load, and (iv) solves a mixed-integer linear program for right-sizing complemented by eight cost-reduction rules. On a twelve-month workload derived from open production traces, the forecasting ensemble attains a mean absolute percentage error of 7.9% at a one-week horizon, and the optimizer identifies \$590.40 of recoverable monthly cost ($\approx 26\%$ of the bill) across 19 recommendations. The results show that combining short-horizon forecasting with commitment-based reservation yields substantial, automated, and explainable savings.

Index Terms—cloud computing, cost optimization, capacity reservation, demand forecasting, right-sizing, reserved instances, mixed-integer programming, FinOps

I. INTRODUCTION

Worldwide public-cloud spending is projected to approach \$600 billion annually [1], and industry surveys consistently report that organizations waste roughly a quarter to a third of that spend on idle or oversized resources [2]. The root cause is a mismatch between *provisioned* capacity and *actual* demand. Cloud providers offer several pricing modes to manage this mismatch: *on-demand* instances, which are flexible but expensive; *reserved* instances (one- or three-year commitments), which are substantially cheaper but rigid; and *spot* instances, which are cheap but interruptible [3].

Choosing among these modes is straightforward when demand is stationary, but real workloads rarely are. A common and costly pattern is a steady *base* demand overlaid with *intermittent random surges*—marketing campaigns, product

launches, seasonal traffic, or batch jobs—whose timing or magnitude is uncertain. Provisioning enough reserved capacity to cover the surges wastes money during the long quiet periods; provisioning only for the base and absorbing surges on-demand pays a premium precisely when usage is highest. Recent operations-research work formalizes this tension as a capacity reservation problem in which a firm meets base demand with standard *base contracts* and copes with surges through flexible *supplementary contracts*, deriving newsvendor-type optimal reservation levels and a threshold purchase/cancel policy [4].

This paper takes a complementary, *systems-and-data* view of the same problem. Rather than assume the demand distribution is known, we *learn* it from telemetry and drive reservation and right-sizing decisions from the resulting forecast. We describe Smart Cloud Optimizer, a deployed platform that turns raw Amazon Web Services (AWS) billing and CloudWatch data into ranked, explainable cost-saving actions. Its central idea mirrors the base/supplementary decomposition of [4]: committed reserved instances serve the forecasted base demand, while elastic capacity sized with a percentile-plus-headroom rule absorbs surges, and a mixed-integer program selects the cheapest feasible instance configuration.

Contributions. This paper makes the following contributions:

- A three-stage architecture (*collect-analyze-optimize*) that integrates multi-service AWS telemetry collection, demand forecasting, and optimization in a single multi-tenant system (Section III).
- A cost model that decomposes demand into a base component served by reserved commitments and a surge component served by on-demand headroom, and a mixed-integer linear program (MILP) for right-sizing under this model (Section IV).
- An anomaly-aware forecasting ensemble of five models, selected by walk-forward cross-validation, together with a rolling-statistics surge detector (Section V).
- An empirical evaluation on a twelve-month workload derived from open production traces, reporting forecasting accuracy and a measured \$590.40 monthly saving

($\approx 26\%$) with full breakdown (Section VI).

II. RELATED WORK

Capacity reservation as an optimization problem. The economics of reserved versus on-demand capacity have been studied extensively in operations research. Chen *et al.* [4] model intermittent random demand surges and characterize an optimal newsvendor reservation level plus a two-threshold contract-management policy; Wang *et al.* [5] study cloud brokerage that reserves in bulk and resells, and Mitrani [6] optimizes server-farm provisioning under power and performance constraints. These works assume the demand distribution is given analytically. Our system instead estimates demand empirically from telemetry and feeds the estimate into the reservation decision, trading closed-form optimality for deployability on real accounts.

Cloud demand and workload forecasting. Predicting cloud workloads is a mature area. Calheiros *et al.* [7] apply ARIMA to workload prediction and quantify the QoS impact; Khan *et al.* [8] use multiple time-series models for co-clustered workloads; and Roy *et al.* [9] drive predictive autoscaling from forecasts. Large-provider studies such as Resource Central [10] predict VM behavior at datacenter scale. We adopt established forecasting models (Holt-Winters/ETS [11], SARIMA [12], and Prophet [13]) but target the *cost* signal directly and use the forecast for reservation sizing rather than autoscaling.

Right-sizing, anomaly detection, and FinOps. Autoscaling and right-sizing techniques are surveyed in [14]. The emerging FinOps discipline [15] emphasizes continuous, accountable cost management. Anomaly detection on cloud metrics is benchmarked by the Numenta Anomaly Benchmark [16]. Our contribution is not a new forecasting or detection algorithm but a system that *composes* forecasting, anomaly filtering, and mixed-integer optimization into an explainable cost-reduction pipeline aligned with the base/supplementary reservation framing of [4].

III. SYSTEM MODEL AND ARCHITECTURE

Smart Cloud Optimizer follows a three-stage pipeline—*collect*, *analyze*, and *optimize*—over a single persistent store (Fig. 1). The design favors a single SQLite database (ACID, *user_id*-isolated multi-tenancy, upsert-based deduplication) so that every stage reads and writes through one typed gateway.

A. Data Collection

A collector pulls data from AWS Cost Explorer, CloudWatch, and the Pricing API, plus per-service inventory for EC2, RDS, S3, Lambda, DynamoDB, EBS, ElastiCache, ECS, ELB, and NAT gateways. Collection proceeds month-by-month over the trailing twelve months to respect API time-window limits. In production (“real mode”) the collector assumes a customer-provided cross-account IAM role via STS; only the role ARN is stored, never long-lived secrets. A “demo mode” serves pre-loaded data for evaluation without credentials. Telemetry is normalized into 30 relational tables keyed

by (*timestamp*, *user_id*, *resource_id*), with the daily total-cost series serving as the primary forecasting target.

B. Demand Model

Let d_t denote the realized demand (here, daily cost) on day t . We model it as the sum of a slowly varying base component, a deterministic weekly seasonal component, and an intermittent surge component:

$$d_t = \underbrace{b_t}_{\text{base}} + \underbrace{s_t}_{\text{seasonal}} + \underbrace{u_t}_{\text{surge}} + \varepsilon_t, \quad (1)$$

where u_t is zero on most days and takes large positive values during surges, and ε_t is noise. The base and seasonal parts are predictable and should be served by cheap committed capacity; the surge part is, by construction, hard to predict in timing and magnitude and must be served by flexible capacity held in reserve as *headroom*. This decomposition is the data-driven analogue of the base/supplementary contract split of [4].

IV. PROBLEM FORMULATION

A. Cost Objective

Consider a fleet of resources $i \in \mathcal{I}$ (e.g., EC2 and RDS instances). Each resource can be assigned a configuration c from a catalog $\mathcal{C}_i$ of candidate instance types, each with an hourly on-demand price p_c^{od} and, where eligible, a reserved price $p_c^{\text{ri}} < p_c^{\text{od}}$. Let H be the monthly hour count. We minimize the total monthly cost subject to meeting demand with surge headroom. Writing the binary assignment variable $x_{i,c} \in \{0, 1\}$,

$$\min_x \sum_{i \in \mathcal{I}} \sum_{c \in \mathcal{C}_i} H \pi_{i,c} x_{i,c} \quad (2)$$

$$\text{s.t.} \quad \sum_{c \in \mathcal{C}_i} x_{i,c} = 1, \quad \forall i, \quad (3)$$

$$\sum_{c \in \mathcal{C}_i} \text{vcpu}_c x_{i,c} \geq \rho_i^{\text{cpu}}, \quad \forall i, \quad (4)$$

$$\sum_{c \in \mathcal{C}_i} \text{mem}_c x_{i,c} \geq \rho_i^{\text{mem}}, \quad \forall i, \quad (5)$$

$$\sum_i \sum_c H \pi_{i,c} x_{i,c} \leq B, \quad (6)$$

where $\pi_{i,c} = p_c^{\text{ri}}$ if resource i qualifies for a reservation (steady, long-running) and p_c^{od} otherwise, and B is an optional budget cap. Constraint (3) assigns exactly one type per resource; (4)–(5) enforce sufficient capacity.

B. Surge-Aware Demand Sizing

The right-hand sides ρ_i^{cpu} and ρ_i^{mem} encode the surge headroom. Rather than sizing to the average (which hides spikes) or the maximum (a single outlier), we size to the 95th percentile of observed utilization scaled by a headroom factor γ :

$$\rho_i^{\text{cpu}} = \frac{P_{95}(\text{CPU}_i)}{100} \cdot \text{vcpu}_i^{\text{cur}} \cdot \gamma, \quad \gamma = 1.3. \quad (7)$$

The factor $\gamma = 1.3$ reserves 30% of capacity as surge headroom, the data-driven counterpart of a supplementary

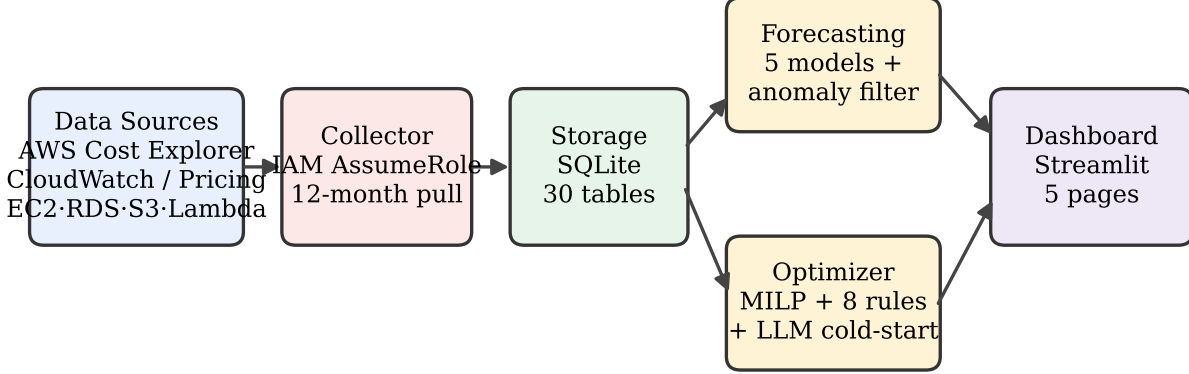


Fig. 1. Three-stage architecture. A collector assumes a cross-account IAM role and pulls twelve months of cost and utilization data into a single store; the forecasting and optimization engines read from it, and a dashboard presents ranked recommendations.

TABLE I
NOTATION

Symbol	Meaning
d_t	realized demand (daily cost) on day t
b_t, s_t, u_t	base, seasonal, surge components
$x_{i,c}$	assign config. c to resource i (binary)
$p_c^{\text{od}}, p_c^{\text{ri}}$	on-demand / reserved hourly price
$\pi_{i,c}$	effective price (reserved if eligible)
$\rho_i^{\text{cpu}}, \rho_i^{\text{mem}}$	required vCPU / memory (sized)
γ	surge headroom factor (1.3)
B	optional monthly budget cap

reservation: enough elastic capacity to absorb the surge term u_t in (1) without paying for it year-round. Table I summarizes the notation.

The MILP (2)–(6) is solved per service with the CBC solver via PuLP [17]. A resource is recommended for change only if the optimal configuration is cheaper than the current one; infeasible instances (demand exceeding the largest catalog type) are reported and skipped.

V. METHODOLOGY

A. Demand Forecasting

The analyze stage forecasts the daily-cost series to estimate the base demand $b_t + s_t$ that justifies reserved commitments. We use an ensemble of five models behind a common fit/predict interface: *Naive* (last value), *Seasonal Naive* (value seven days prior), *ETS* (Holt-Winters with additive weekly seasonality) [11], *Prophet* [13], and auto-tuned *SARIMA* [12]. Each model emits a point forecast and a 95% interval. Model choice is data-dependent: with little history a robust baseline is preferred, and the system selects ETS once at least two months of data are available (Section VI).

B. Surge Detection and Anomaly Filtering

Before fitting, the series is cleaned of surges so that anomalous spikes do not bias the base-demand estimate. We flag day t as a surge if it deviates from its trailing window by more than three standard deviations (rolling z -score, window 30) or falls outside the interquartile fence $[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$; the two detectors are combined by union [16]. Flagged points are excluded from training but retained for surge characterization, since they define the headroom the optimizer must provision.

C. Walk-Forward Model Selection

Models are compared by expanding-window (walk-forward) cross-validation: from an initial training window the model forecasts a fixed horizon, the window advances by a fixed step, and a fresh model is refit each fold. We report mean absolute percentage error (MAPE), root-mean-square error (RMSE), and mean absolute error (MAE); MAPE masks zero-demand days. This procedure respects temporal order and avoids look-ahead leakage.

D. Reservation, Rules, and Cold-Start

Three mechanisms turn forecasts into actions. (1) *Reservation*: long-running resources with at least 60 days of steady utilization are recommended for one-year reserved pricing—committed capacity for the base demand. (2) *Right-sizing*: the MILP of Section IV selects the cheapest surge-feasible configuration. (3) *Rules*: eight threshold heuristics (Table II) cover services for which an LP is unnecessary (storage class, capacity mode, idle deletion, etc.). For new users with no telemetry, an LLM-assisted cold-start module (Gemini 2.5 Flash [18]) maps a short questionnaire about expected traffic pattern and availability to an initial architecture, addressing the cold-start case of reservation planning. A final deduplication keeps, for each resource, the single highest-saving recommendation.

TABLE II
RULE-BASED COST-REDUCTION CHECKS

Service	Trigger	Action
EC2/RDS	on-demand, running ≥ 60 d	reserve (1-yr)
Lambda	avg mem $< 50\%$ alloc	downsize tier
EBS	gp2 / unattached / idle	gp3, delete
S3	STANDARD, < 100 req/day	intelligent-tiering
DynamoDB	provisioned, util $< 50\%$	on-demand mode
NAT	cost $> \$30$ /mo	VPC endpoint
ELB	no targets, no traffic	delete idle

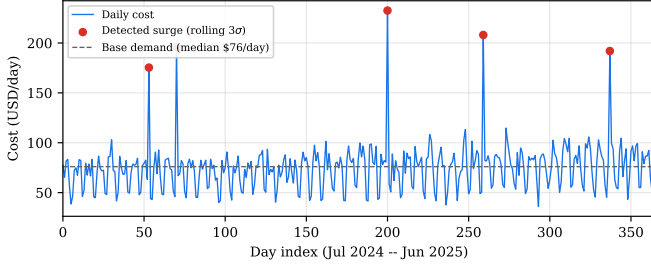


Fig. 2. Daily cost with detected demand surges. A stable base demand is punctuated by intermittent spikes; the optimizer reserves the base and holds 30% headroom for the surges.

VI. EXPERIMENTAL EVALUATION

A. Dataset and Setup

We evaluate on a twelve-month workload (365 daily-cost observations, July 2024– June 2025) for a simulated mid-size SaaS tenant. The workload is constructed from open production traces—the Bitbrains GWA-T-12 VM utilization dataset [19] and the Numenta Anomaly Benchmark [16]—combined with live AWS Pricing API data, so that utilization shapes, anomalies, and prices are realistic. The tenant’s mean cost is \$74.64/day (min \$36.10, max \$232.52), about \$2,239/month. Forecasting uses walk-forward CV; optimization uses the MILP and rules of Sections IV–V with $\gamma = 1.3$. All figures and numbers are reproduced by `paper/make_figures.py` against the shipped database.

B. Surge Characterization

Fig. 2 shows the daily-cost signal with surges flagged by the rolling- z -score detector. The series exhibits a stable base around the median of \$70/day punctuated by sharp, short-lived spikes reaching \$232/day—the intermittent random surges that motivate headroom-based provisioning. The detector isolates these spikes cleanly, leaving a base series suitable for reservation sizing.

C. Forecasting Accuracy

Table III and Fig. 3 report MAPE by horizon from walk-forward CV. Prophet is most accurate at short horizons (MAPE 7.9% at 7 days, 9.8% at 14 days), while ETS dominates the medium range (10–90 days), and the seasonal baseline becomes competitive only at very long horizons where all

TABLE III
FORECASTING MAPE (%) BY HORIZON (WALK-FORWARD CV)

Horizon (d)	Naive	Seas. Naive	ETS	Prophet
7	40.5	16.0	9.2	7.9
14	22.9	17.9	10.5	9.8
30	25.7	14.6	10.4	9.5
60	25.5	14.9	10.8	22.2
90	25.8	13.1	12.0	12.4
180	43.1	16.0	12.9	27.9
300	32.2	16.8	24.0	34.1

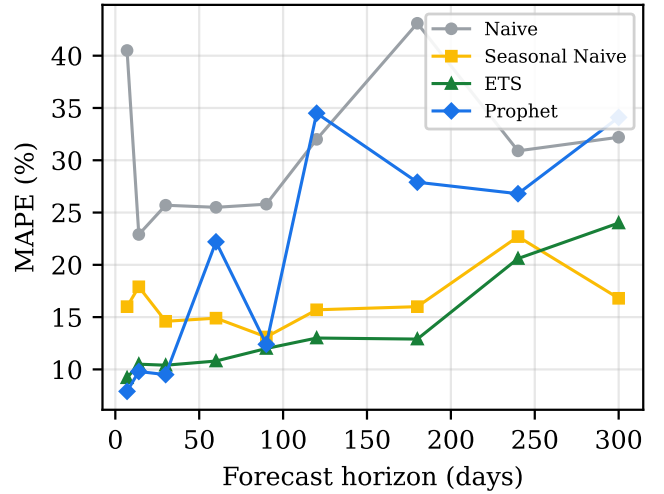


Fig. 3. Forecasting error versus horizon. Prophet leads at short horizons; ETS is most robust through the mid-range relevant to reservation planning.

models degrade. Because reservation decisions hinge on near-term base demand, the short-horizon regime is the operative one, and there the ensemble achieves single-digit error. Fig. 4 shows a representative 30-day holdout in which the deployed seasonal forecaster tracks the base demand at 11.9% MAPE while the 95% interval contains the realized surges.

D. Optimization Savings

Running the optimizer over the tenant yields 19 recommendations totaling **\$590.40/month**, about 26% of the \$2,239 monthly bill. Fig. 5 and Table IV break the savings down by service and action. The two largest items are RDS right-sizing (\$362.81/month combined; a production PostgreSQL instance running at a fraction of its provisioned size is reduced, saving 93.3% on that resource) and EC2 reservation (\$208.42/month from switching six steady, long-running instances from on-demand to one-year reserved pricing at $\approx 37\%$ unit discount). Smaller but safe wins—deleting unattached EBS volumes, upgrading gp2 to gp3, replacing costly NAT data paths with VPC endpoints, tiering cold S3 objects, and downsizing an over-allocated Lambda—account for the remainder. Every recommendation carries a confidence label and a human-readable rationale, so an operator can audit it before applying.

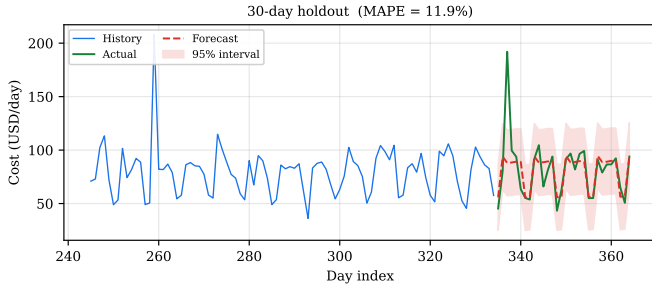


Fig. 4. A 30-day holdout forecast of daily cost. The forecast tracks base demand at 11.9% MAPE and the 95% interval brackets the surges.

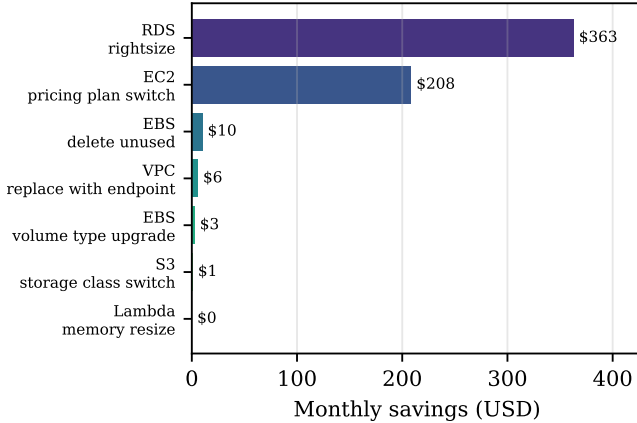


Fig. 5. Monthly savings by service and action. Reservation (EC2) and right-sizing (RDS) dominate, consistent with the base/surge cost model.

E. Discussion and Threats to Validity

The results confirm the central hypothesis: serving forecasted base demand with reserved commitments while holding percentile-based headroom for surges recovers roughly a quarter of the bill without risking surge SLAs. The dominant savings come precisely from the two levers of the base/surge model—reservation and surge-aware right-sizing—validating the framing borrowed from [4]. Two threats to validity should be noted. First, the evaluation uses a workload synthesized from open traces rather than a live production account; while utilization shapes and prices are realistic, absolute savings will vary by tenant. Second, the forecasting accuracy is measured on the cost signal of a single tenant; broader validation across heterogeneous accounts is future work. Neither threat affects the system’s mechanism, which is account- and dataset-agnostic.

VII. CONCLUSION AND FUTURE WORK

We presented Smart Cloud Optimizer, a forecast-driven system for cloud cost optimization under intermittent random demand surges. By decomposing demand into a reservable base and a surge component, forecasting the base with an anomaly-aware ensemble, and right-sizing with a surge-aware MILP, the system recovers \$590.40/month ($\approx 26\%$) on a realistic twelve-

TABLE IV
RECOMMENDATION BREAKDOWN (MONTHLY SAVINGS)

Service	Action	Count	Savings (\$)
RDS	right-size	2	362.81
EC2	reserve (1-yr)	6	208.42
EBS	delete unused	3	10.00
VPC	replace with endpoint	2	5.67
EBS	gp2 $\rightarrow$ gp3	4	2.60
S3	storage-class switch	1	0.79
Lambda	memory resize	1	0.11
Total		19	590.40

month workload while forecasting near-term demand at single-digit MAPE. The work operationalizes the base/supplementary reservation theory of [4] in a deployable data pipeline. Future work includes incorporating spot instances and a threshold-based renew/cancel policy for reservations, validating on live multi-tenant accounts, and closing the loop by automatically applying and monitoring the recommended actions.

ACKNOWLEDGMENT

The authors thank the Faculty of Computer and Information Sciences, Ain Shams University, for support, and acknowledge the providers of the Bitbrains GWA-T-12 and Numenta Anomaly Benchmark datasets.

REFERENCES

- [1] Gartner, Inc., “Gartner forecasts worldwide public cloud end-user spending to reach nearly \$600 billion,” *Gartner Press Release*, 2023.
- [2] Flexera, “State of the cloud report,” *Flexera*, 2024.
- [3] Amazon Web Services, “Amazon EC2 pricing and reserved instances,” <https://aws.amazon.com/ec2/pricing/>, 2024.
- [4] S. Chen, J. Lei, and K. Moinszadeh, “Cost optimization in cloud computing: Capacity reservation for intermittent random demand surges,” *Production and Operations Management*, vol. 33, no. 6, pp. 1265–1284, 2024.
- [5] W. Wang, D. Niu, B. Li, and B. Liang, “Dynamic cloud resource reservation via cloud brokerage,” in *Proc. IEEE 33rd Int. Conf. Distributed Computing Systems (ICDCS)*, 2013, pp. 400–409.
- [6] I. Mittrani, “Managing performance and power consumption in a server farm,” *Annals of Operations Research*, vol. 202, no. 1, pp. 121–134, 2013.
- [7] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, “Workload prediction using ARIMA model and its impact on cloud applications’ qos,” *IEEE Transactions on Cloud Computing*, vol. 3, no. 4, pp. 449–458, 2015.
- [8] A. Khan, X. Yan, S. Tao, and N. Anerousis, “Workload characterization and prediction in the cloud: A multiple time series approach,” *Proc. IEEE Network Operations and Management Symp. (NOMS)*, pp. 1287–1294, 2012.
- [9] N. Roy, A. Dubey, and A. Gokhale, “Efficient autoscaling in the cloud using predictive models for workload forecasting,” in *Proc. IEEE 4th Int. Conf. Cloud Computing (CLOUD)*, 2011, pp. 500–507.
- [10] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proc. 26th Symp. Operating Systems Principles (SOSP)*, 2017, pp. 153–167.
- [11] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021.
- [12] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.
- [13] S. J. Taylor and B. Letham, “Forecasting at scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

- [14] T. Lorigo-Botran, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.
- [15] J. R. Storment and M. Fuller, "Cloud FinOps: Collaborative, real-time cloud financial management," *O'Reilly Media*, 2023.
- [16] A. Lavin and S. Ahmad, "Evaluating real-time anomaly detection algorithms – the numenta anomaly benchmark," in *Proc. 14th IEEE Int. Conf. Machine Learning and Applications (ICMLA)*, 2015, pp. 38–44.
- [17] S. Mitchell, M. O'Sullivan, and I. Dunning, "PuLP: A linear programming toolkit for Python," *The University of Auckland*, 2011.
- [18] Google DeepMind, "Gemini 2.5: Multimodal models for reasoning and generation," <https://deepmind.google/technologies/gemini/>, 2024.
- [19] S. Shen, V. van Beek, and A. Iosup, "Statistical characterization of business-critical workloads hosted in cloud datacenters," in *Proc. 15th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGrid)*, 2015, pp. 465–474.